Software Re-Engineering

Lecture 6

Declarative and scripted file in Jenkins

Instructor:

Dr. Natalia Chaudhry,

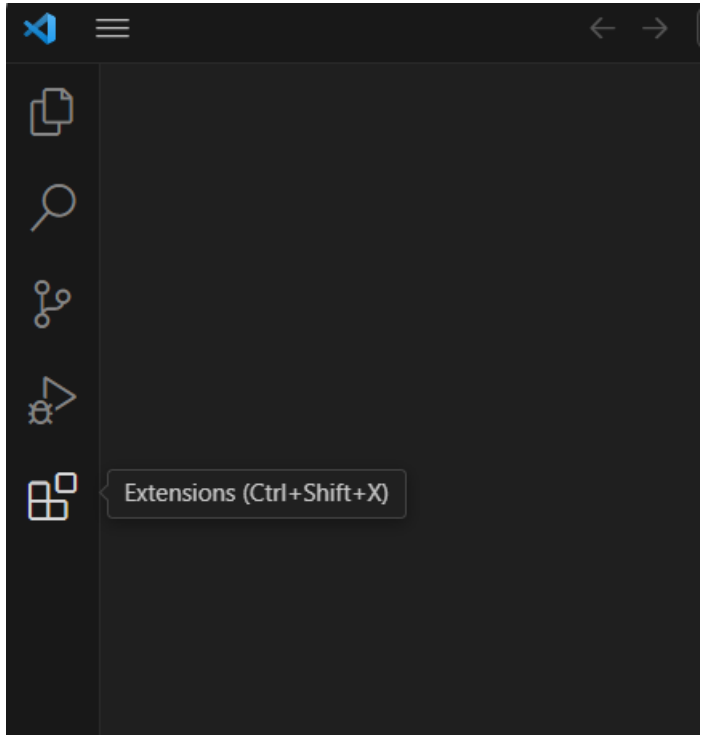
Assistant Professor, Software Engineering Department, PU.

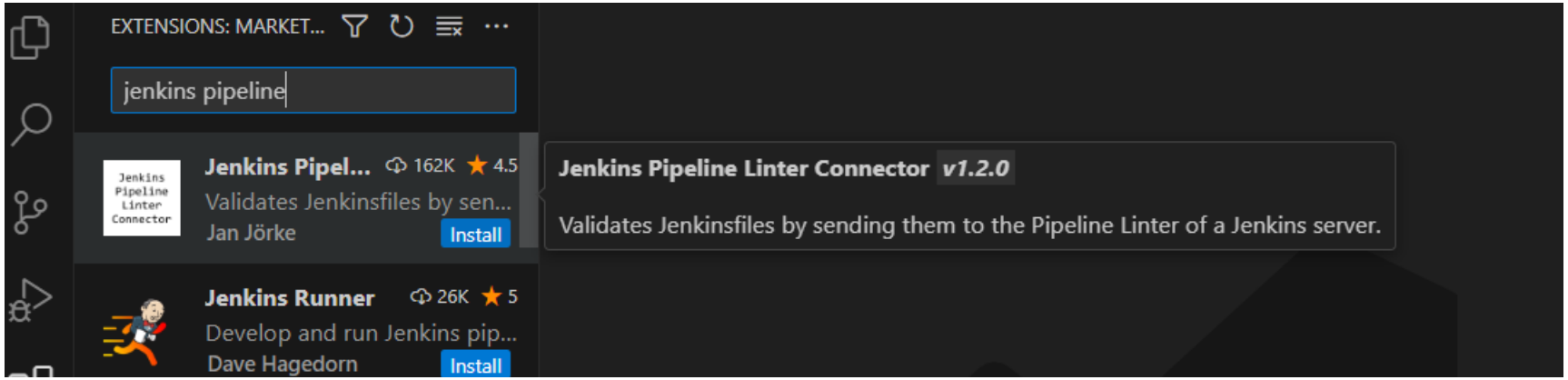
Agenda

- Declarative and scripted file in Jenkins
- Rearchitecturing and related concepts

VS Code IDE for writing Jenkins pipeline scripts

- Visual Studio Code is a lightweight but powerful source code editor which runs on your desktop and is available for Windows, macOS and Linux.

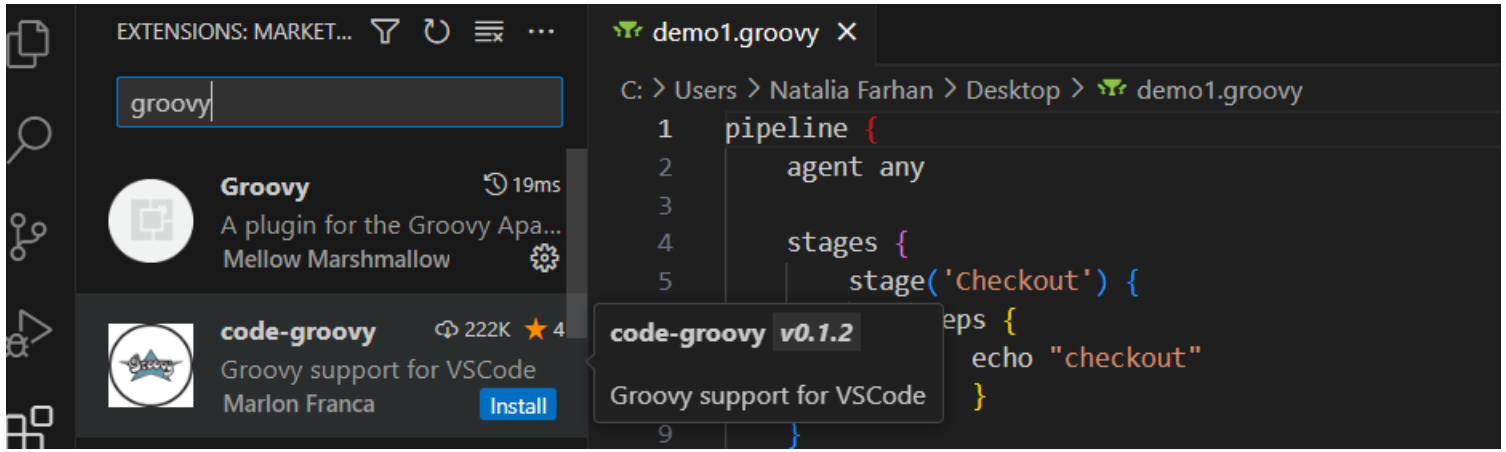



EXTENSIONS: MARKET...
jenkins pipeline
Jenkins Pipel... 162K 4.5
Validates Jenkinsfiles by sen...
Jan Jörke
Install
Jenkins Runner 26K 5
Develop and run Jenkins pip...
Dave Hagedorn
Install
Jenkins Pipeline Linter Connector v1.2.0
Validates Jenkinsfiles by sending them to the Pipeline Linter of a Jenkins server.

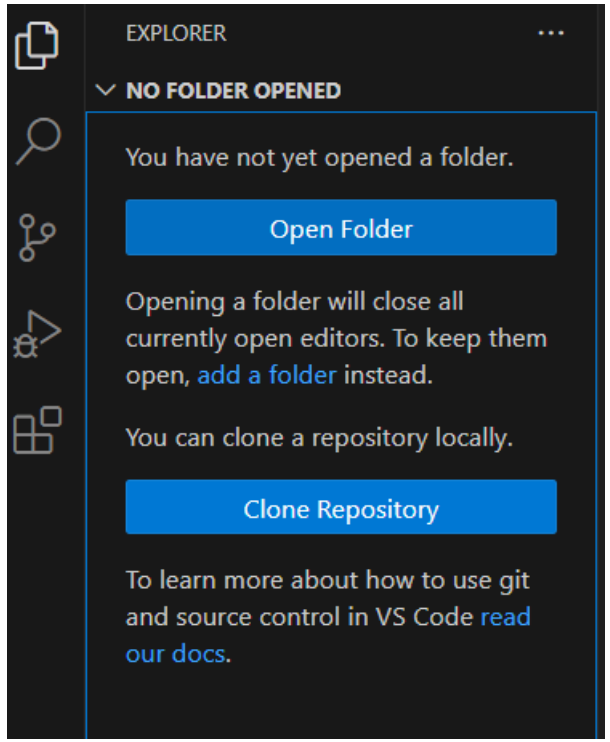
```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                echo "checkout"
                }
    }
}
```

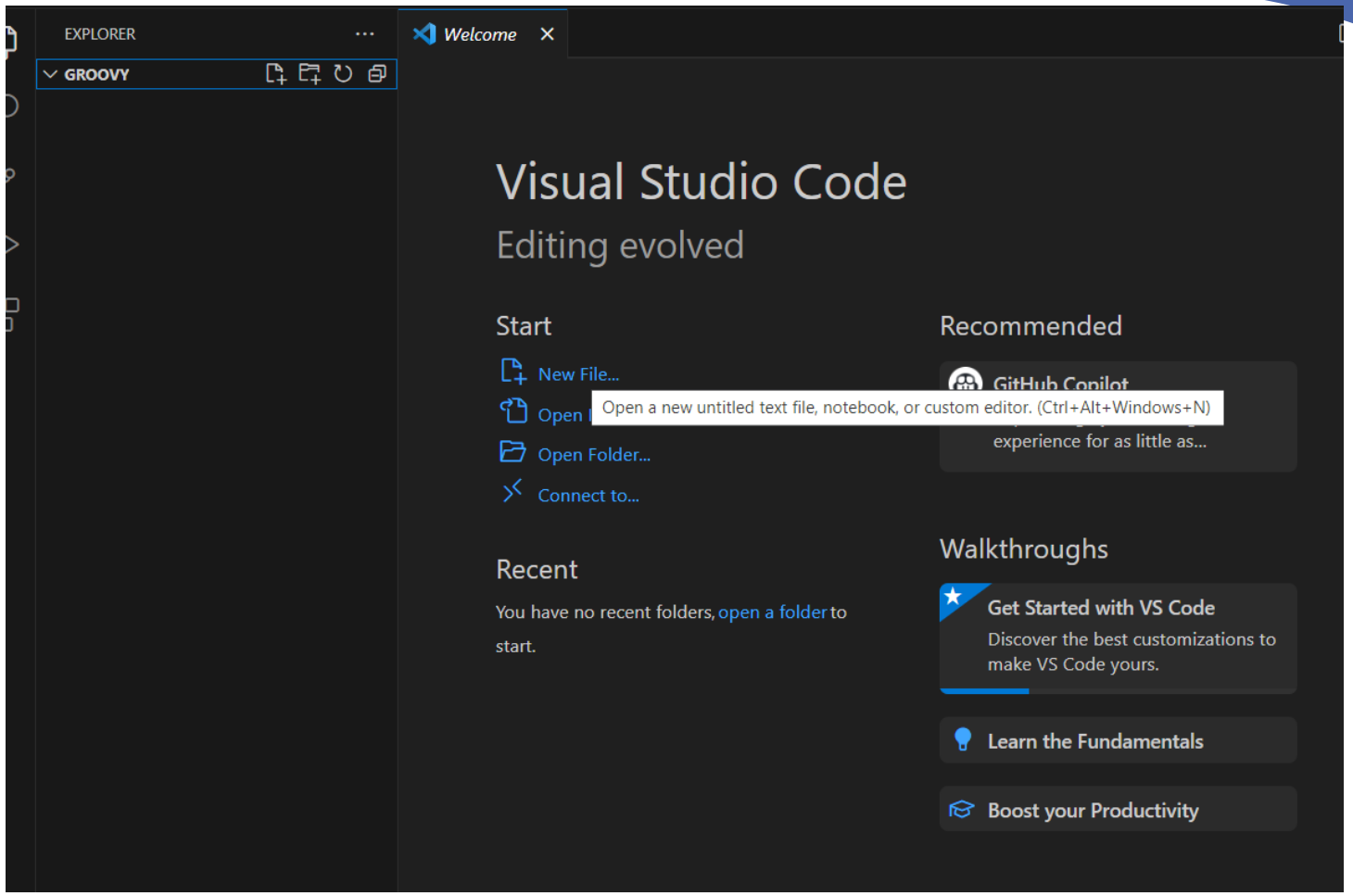
Install groovy plugin(code-groovy) to run and debug

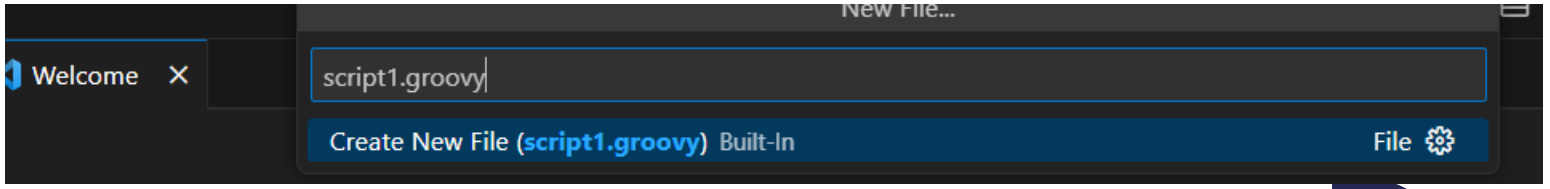


Create a folder o desktop and select **Open a folder**



New file


EXPLORER
GROOVY
Welcome
Visual Studio Code
Editing evolved
Start
New File...
Open
Open Folder...
Connect to...
Recommended
GitHub Copilot
Open a new untitled text file, notebook, or custom editor. (Ctrl+Alt+Windows+N)
experience for as little as...
Recent
You have no recent folders, open a folder to start.
Walkthroughs
Get Started with VS Code
Discover the best customizations to make VS Code yours.
Learn the Fundamentals
Boost your Productivity


New File...
Welcome
script1.groovy
Create New File (script1.groovy) Built-In
File

Install groovy

go to the official site of Groovy. Below is the link:

https://groovy.apache.org/

The Groovy page will be open. Now click on the Download link.

Now, click on the Groovy 3.0 Windows Installer to download the Groovy.

If error then Set env var

Right Click on This PC>>go to Properties>>Go to Advance System Setting(at Left)

In This Window

Go to Environment Variables

Click on New Enter variable name 'groovy'.

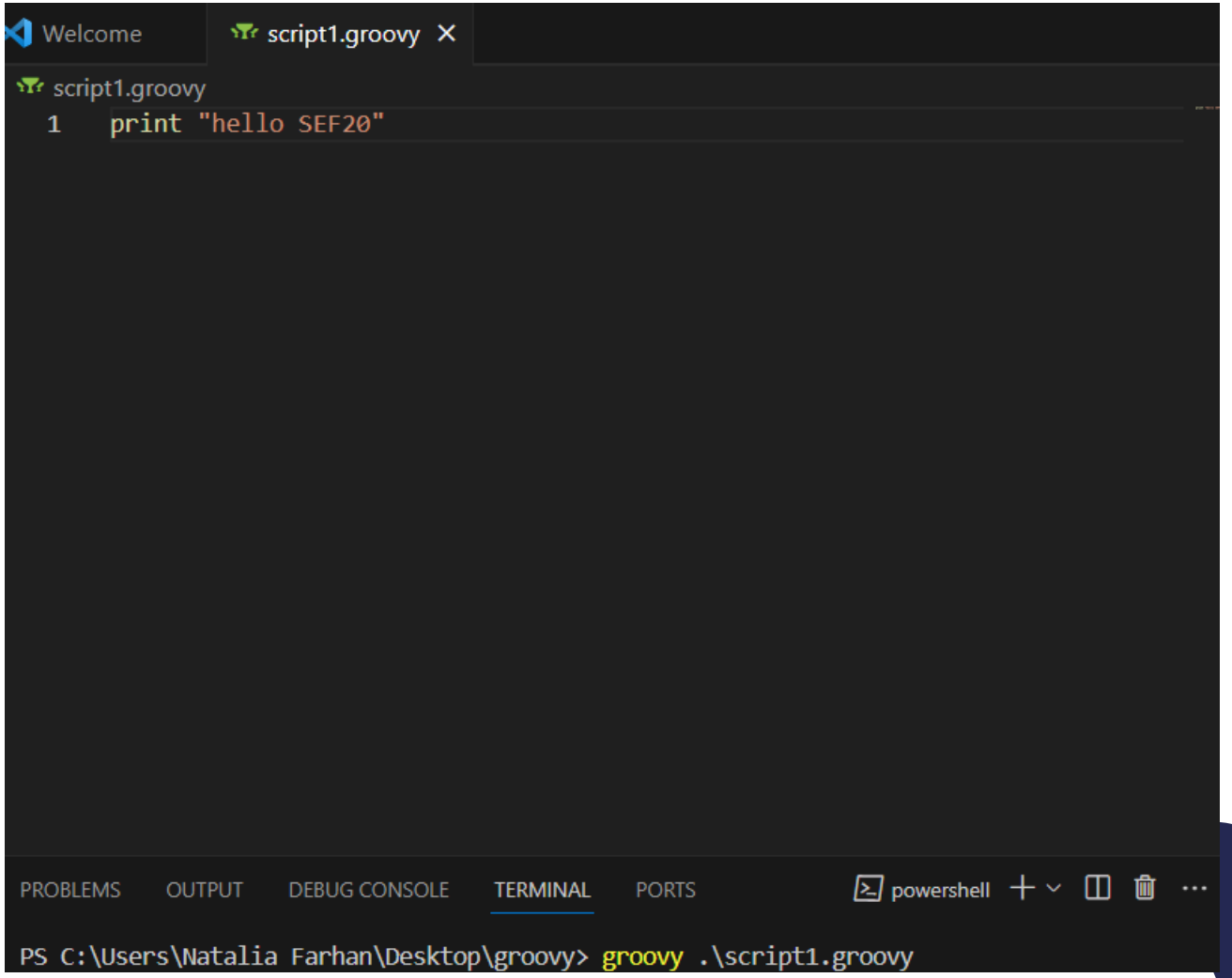
For path goto this pc and select the path where groovy is installed for Example::

C:\Program Files (x86)\Groovy\bin

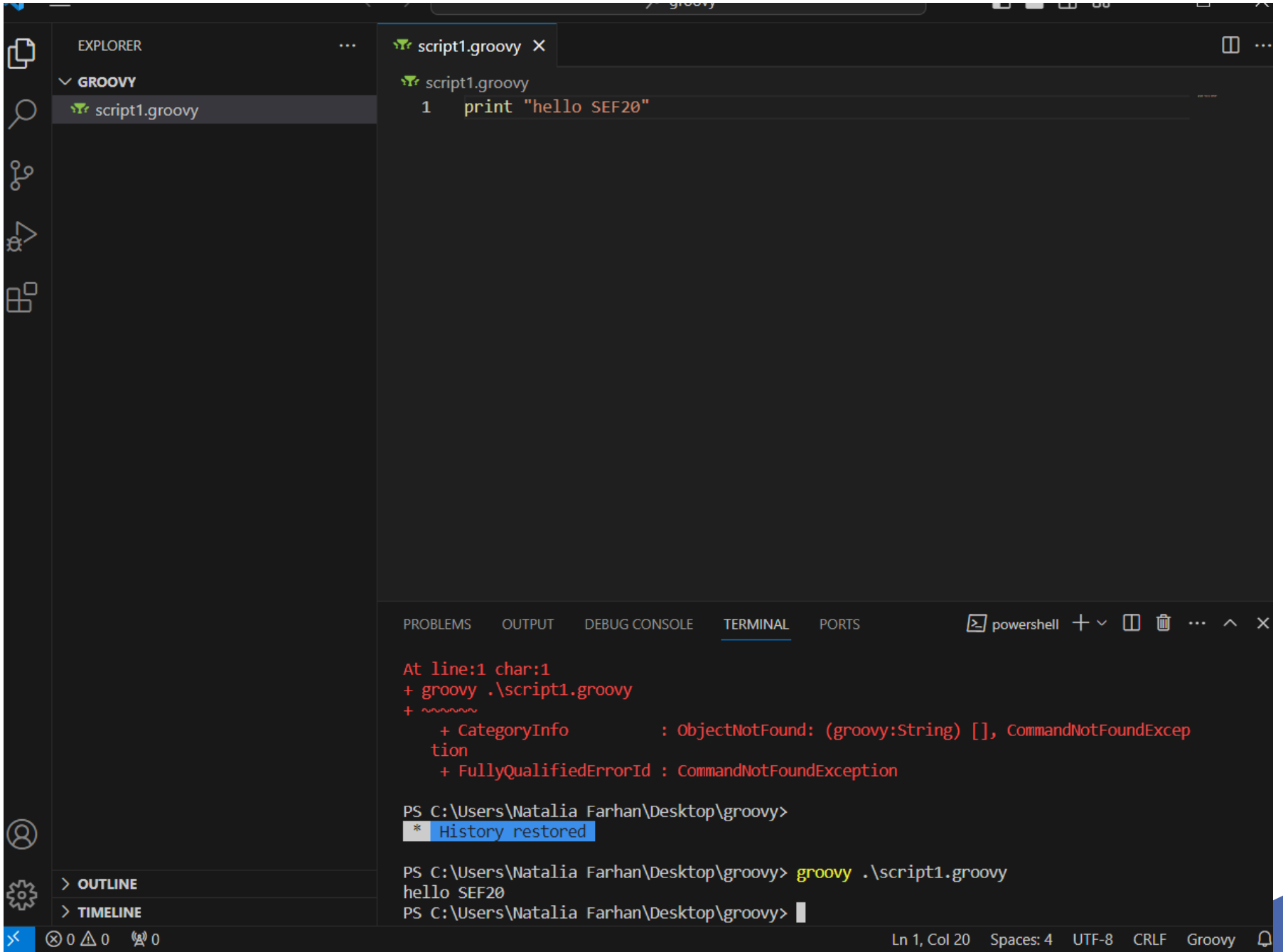
now press save and ok. Restart your VS Code editor..

Select **View-> Terminal**

Type **Groovy .\script.groovy** and **Enter**

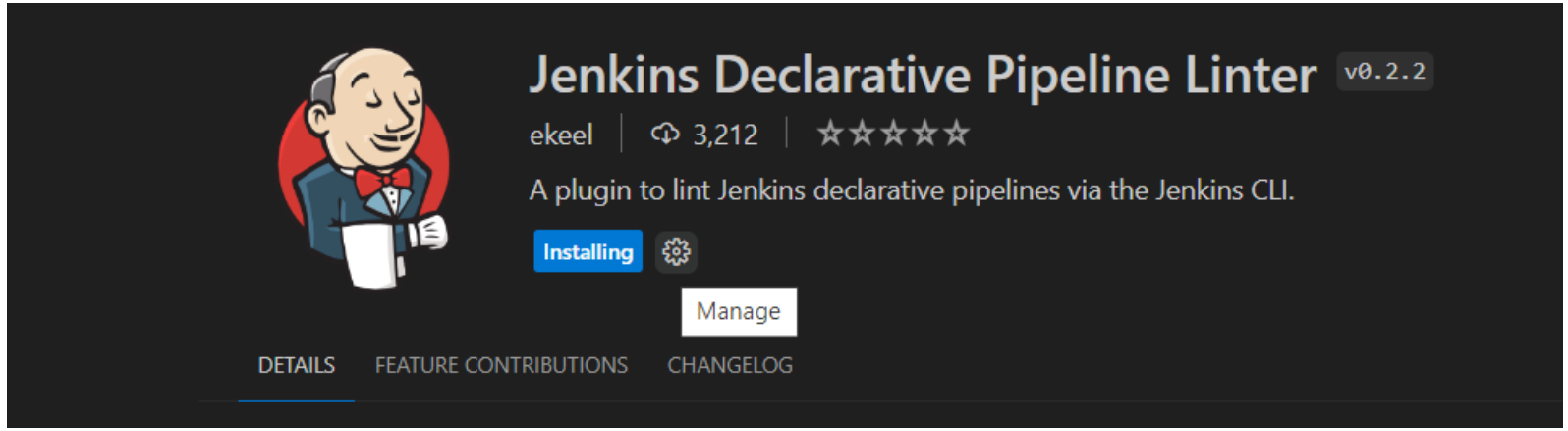


Restart the system

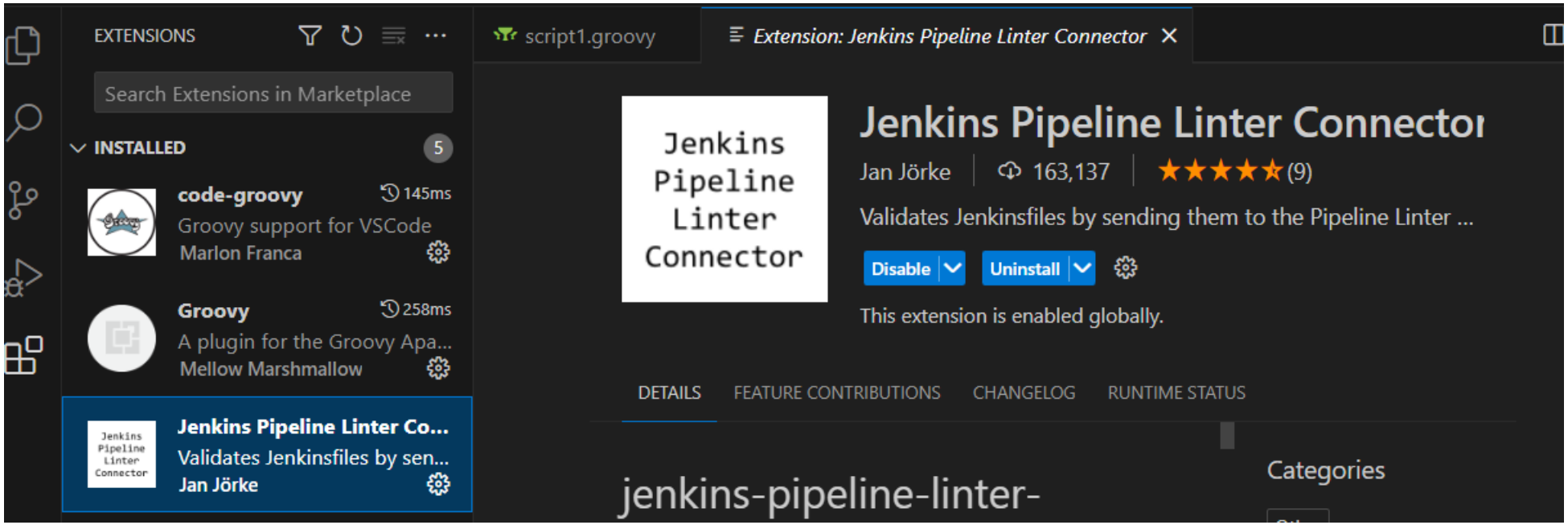


Thank you

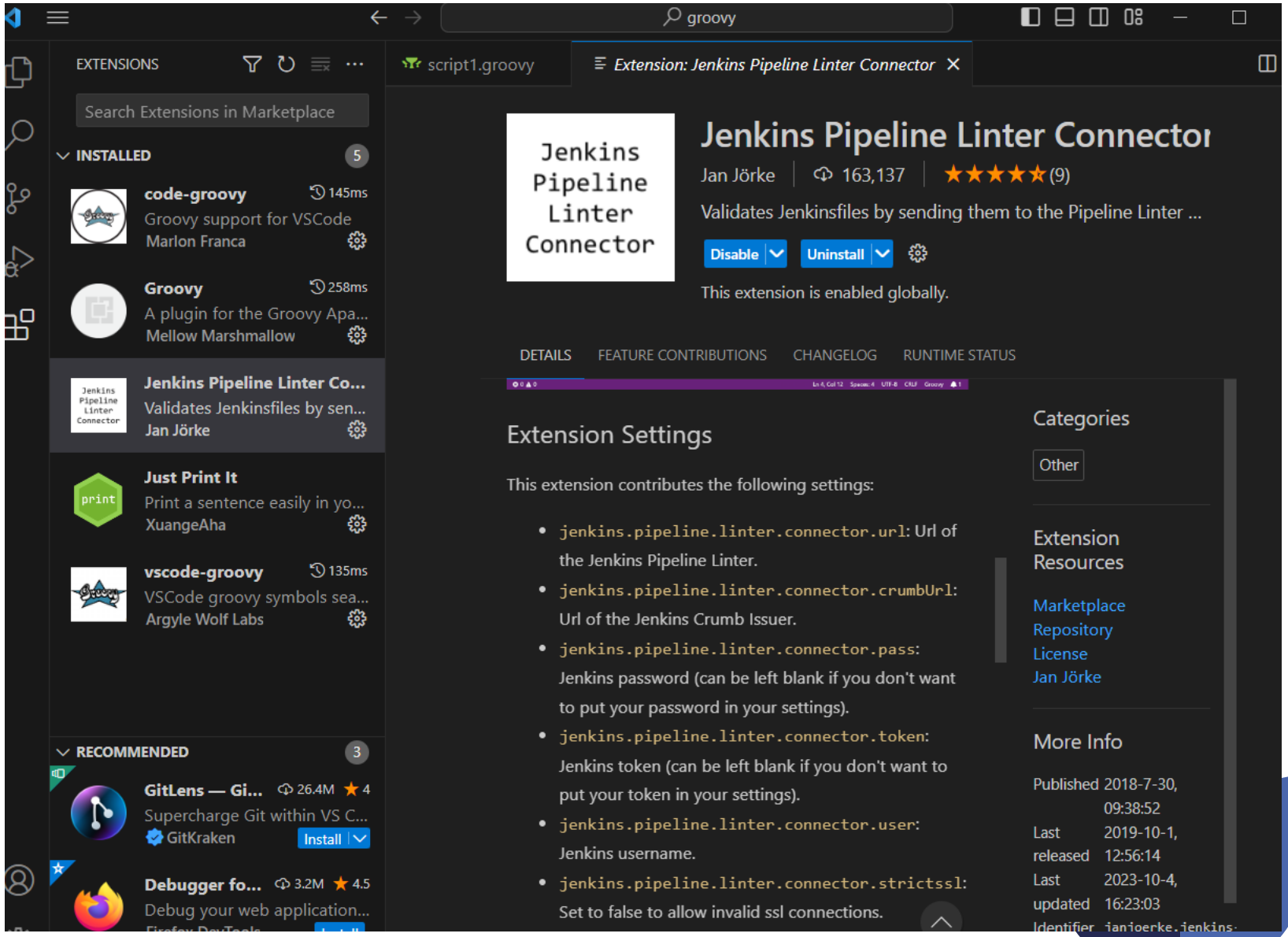
Install Jenkins declarative pipeline plugin



Validator plugin

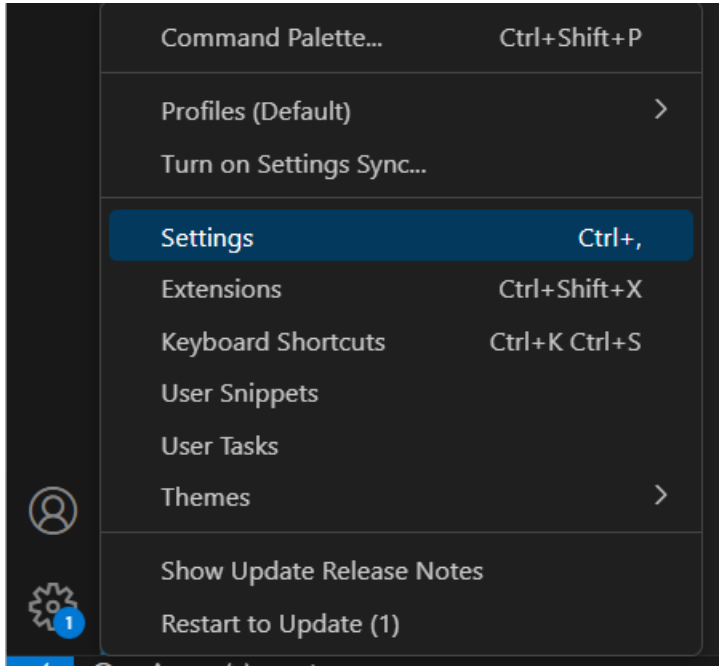


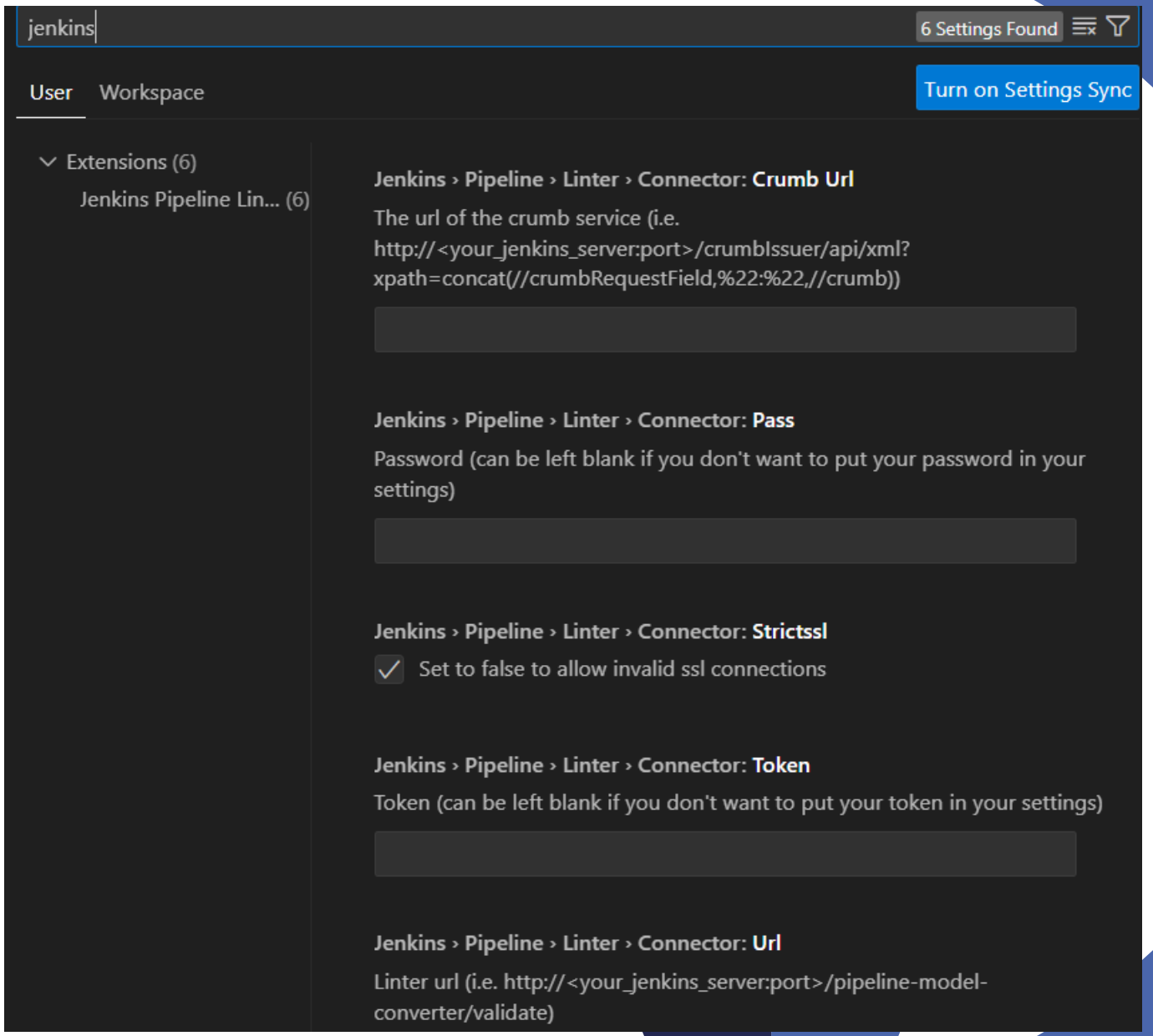
Search for details of a relevant plugin



Go to settings

Search for **Jenkins**





Set appropriate fields from information got before (beneath each plugin details)

Jenkins › Pipeline › Linter › Connector: Url

http://localhost:8080/pipeline-model-converter/validate

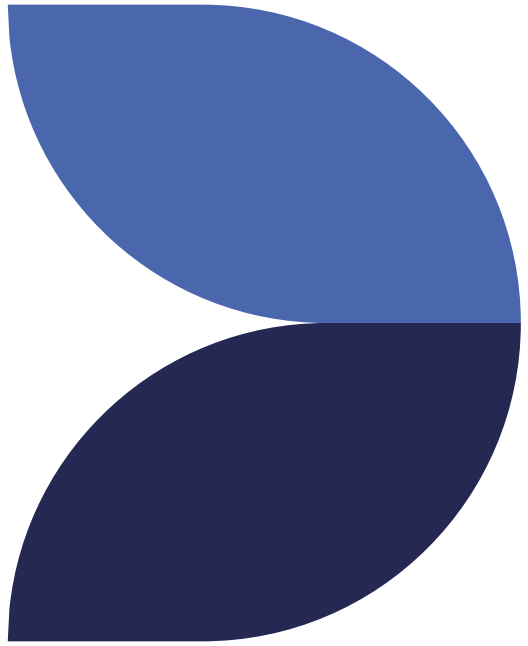
```
pipeline {
    agent any
    stages{
        stage('build')
        {
            parallel{
                stage('Build'){
                    steps{
                        echo 'building...'
                    }
                }
                stage('Test'){
                    steps{
                        echo 'testingf...'
                    }
                }
                stage('Test log'){
                    environment{
                        localVariable='HelloLocal'
                    }

                }

            }

        }
        stage('deploy')
        {
            parallel{
                when{
                    branch 'master'
                }
            }
            stage('deploy')
            {
                steps{
                    input(message: 'Do you want deployment?',id:'ok')
                    echo 'deploying app'
                }
            }
        }
    }
}
```

Declarative



Agent

Jenkins is popular for one of its best features called distributed build process to the agent nodes. Jenkins build pipeline plugin ensures the same feature present in the pipeline that are created in the Declarative method. Normally, the agent will be declared at the top-level as a global agent declaration. So, the Agent label is ideally mentioned with the following parameters.

any – This means pipeline will run in any available node. (agent any)
none – Ideally None mean, the pipeline will run in not run in any agent at top-level, but each stage should be defined with its own agent (agent none).
label – This means the pipeline will be mentioned as label name and pipeline will look for the available node with the label name mentioned (agent {label ‘my label name for node’})
node – mentioning node in the agent is same as mentioning label but this will give us more option like custom Workspace (agent {node{label ‘my label name’}}).
docker – By mentioning docker, the pipeline will run in a docker environment. (agent{ docker {image ‘ruby3:develop’ label ‘my label name’ arg ‘—port 3000:80’} }))

Steps

A stage can have multiple commands to run for completing the continuous delivery process. basically, Step is the set of commands that performs an action and It will be executed one by one.

Environment (Defined at stage or pipeline level)

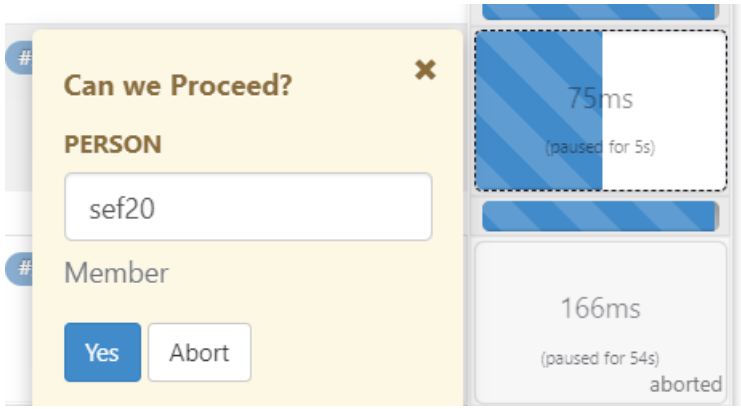
The environment is the directive that contains the Key-value pairs of the environment variable that should be available for the steps that are going to be executed in the stages.

Input (Defined at stage level)

Input Directive is declared inside stage and it will help us getting input by prompting users. Basically, The stage section will wait until any Input is submitted. So, the following are the configuration options.

- **Message** – This is to show some message to the user
- **ID** – This is the Default value
- **Ok** – This is the “OK” Button
- **Submitter** – is the comma-separated value list of users who are allowed to enter the input.
- **submitterParameter –** This will add submitter’s name into the environment variable
- **parameters –** This is the list of parameters that need to be provided by the submitter.Line 1

```
pipeline {
    agent any
    stages {
        stage('Example') {
            input {
                message "Can we Proceed?"
                ok "Yes"
                submitter “SEF20"
                parameters {
                    string(name: 'PERSON', defaultValue: ‘SEF20',
description: 'Member')
                }
            }
            steps {
                echo "${PERSON}, is proceeding..."
            }
        }
    }
}
```


Can we Proceed?
PERSON
sef20
Member
Yes
Abort
75ms
(paused for 5s)
166ms
(paused for 54s)
aborted

When

“When” is the conditional derivate of the Jenkins declarative pipeline. Just like “IF” conditions, “when” will have the one or more condition statement. This can be either for from the user input or from the Built-in conditions.

So, Let us see some built-in conditions.

Branch – This will execute the stage only when the branch in SCM is matching. (when { branch 'master' })
buildingTag – This executes only when the Build is building the tag (when { buildingTag() })
changelog – Stage will run only when the changelog is having a specified pattern in it. (when { changelog '.*^\\[SERVICE2\\] .+$' })
changeset – This will run only when the SCM changeset is having the specified pattern in file (when { changeset "**/*.js" })
changeRequest – ChangeRequest is like the pull request in GitHub and the stage will run only after the change request is raised (when { changeRequest target: 'master' })
environment – Stage will execute only when the specified environment variable is matching (when { environment name: 'DEPLOY_TO', value: 'production' })

When

“When” is the conditional derivate of the Jenkins declarative pipeline. Just like “IF” conditions, “when” will have the one or more condition statement. This can be either for from the user input or from the Built-in conditions.

equals – Executes only when the expected value and the actual value is matching in the condition (when { equals expected: 2, actual: currentBuild.number })
expression – Condition is created with groovy expression and expects the Boolean true to pass. (when { expression { return params.DEBUG_BUILD } })
tag – Stage will be executed only after the specified tag name pattern matches the stage tag (when { tag "release-*" })
not – This is like “NOT” in the “IF” condition. (when { not { branch 'master' } })
allOf – This is for nested condition and this will expect all should be true. This is like “AND” in “IF” Condition (when { allOf { branch 'master'; environment name: 'DEPLOY_TO', value: 'production' } })
anyOf – anyOf is like “OR” in the “IF” condition. (when { anyOf { branch 'master'; branch 'staging' } })
triggeredBy – the stage will execute only when it is triggered by the specified build. (when { triggeredBy 'SCMTrigger' })

```
pipeline {
    agent any
    stages {
        stage('Example Build') {
            when {
                anyOf {
                                        branch 'master'; branch
'staging'
                                }
            }
            steps {
                echo 'Hello World'
            }
        }
        stage('Example Deploy') {
            when {
                branch 'production'
            }
            steps {
                echo 'Deploying'
            }
        }
    }
}
```

Parallel

Parallel block will allow us to run the stages concurrently. Idealy, Stages that are declared inside the Parallel derivative will run concurrently at the same time. If you want to stop the build process, You need to mention “failFast” true in the parallel section.

```
pipeline {
    agent any
    stages {
        stage('NORMAL Stage') {
            steps {
                echo 'I am one'
            }
        }
        stage('Parallel Stage') {
            when {
                branch 'master'
            }
            failFast true
            parallel {
                stage('stage one') {
                    agent {
                        label "stageonebranch"
                    }
                    steps {
                        echo "Me in stage one"
                    }
                }
                stage('Stage two') {
                    agent {
                        label "stage two"
                    }
                    steps {
                        echo "Me in stage two"
                    }
                }
                stage('Stage three') {
                    agent {
                        label "Stage Three"
                    }
                }
            }
        }
    }
}
```

Parameters

Parameter Directive will provide the trigger parameters to the pipeline. Following are the available Parameters

string – simple parameter in string datatype (parameters { string(name: 'yyyyy', defaultValue: 'XXX', description: 'Hello world') })
text – multiline text datatype (parameters { text(name: 'Demo', defaultValue: '', description: 'Demo parameter')})
booleanparam – Boolean type parameter (booleanParam(name: 'Boolean', defaultValue: true, description: 'Boolean value'))
choice – it is like a multiple-choice parameter (parameters { choice(name: 'CHOICE', choices: ['A', 'B', 'C'], description: ‘Choose one'))
password – Password type parameter (password(name: 'PASSWORD', defaultValue: 'Key', description: 'Enter a password'))

```
pipeline {
    agent any
    parameters {
        string(name: 'yyyyy', defaultValue: 'XXX', description: 'Hello world')

        text(name: 'Demo', defaultValue: '', description: 'Demo parameter')

        booleanParam(name: 'Boolean', defaultValue: true, description: 'Boolean value')

        choice(name: 'CHOICE', choices: ['A', 'B', 'C'], description: ‘Choose one')

        password(name: 'PASSWORD', defaultValue: 'Key', description: 'Enter a password')

        file(name: "FILE", description: "file to upload")
    }
    stages {
        stage('Example') {
            steps {
                echo "Hello ${params.yyyyy}"

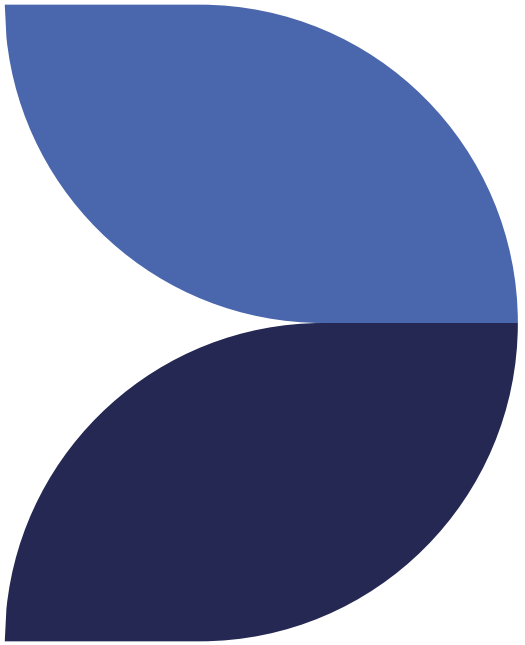
                echo "Biography: ${params.Demo}"

                echo "Toggle: ${params.Boolean }"

                echo "Choice: ${params.CHOICE}"

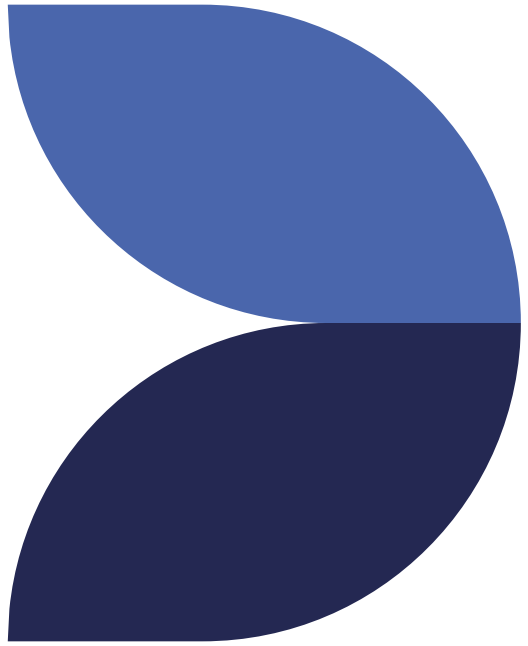
                echo "Password: ${params.PASSWORD}"
            }
        }
    }
}
```

Scripted pipeline



Unlike Declarative, Scripted Pipeline is built with Groovy. Most functionality provided by the Groovy language is made available to users of Scripted Pipeline, which means it can be a very expressive and flexible tool with which one can author continuous delivery pipelines.

Flow control



Scripted Pipeline is serially executed from the top of a Jenkinsfile downwards, like most traditional scripts in Groovy or other languages. Providing flow control, therefore, rests on Groovy expressions, such as the if/else conditionals, for example:

```
node {
    stage('Example') {
        if (env.BRANCH_NAME == 'master') {
            echo 'I only execute on the master
branch'
        } else {
            echo 'I execute elsewhere'
        }
    }
}
```

```
node {
    stage('Example') {
        try {
            sh 'exit 1'
        }
        catch (exc) {
            echo 'Something failed, I should sound the klaxons!'
            throw
        }
    }
}
```